



Presenter 1 — Introduction & research

Slides 1 – 7 · target time ≈ 8 minutes

Your name: _____ Roll no.: _____

You take over from: — (you open the presentation) · You hand over to: Presenter 2 (architecture)

How to use this script

The **Say** text is written to be spoken at a natural pace (about 130 words a minute). You do not need to read it word for word: learn the first and last sentence of each slide and the numbers, and speak the middle in your own words. *Italic cues* tell you what to point at. Everything technical here matches the code in the repository.

The whole talk at a glance

Part	Presenter	Slides	Time	What it covers
1	Presenter 1	1 – 7	≈ 8 min	Title, abstract, introduction, problem & objectives, literature review (2 slides), research gap
2	Presenter 2	8 – 11	≈ 8 min	System architecture, install flow, attestation & logging flow, cryptography & the Warp log
3	Presenter 3	12 + live demo A	≈ 9 min	Heddle sandbox & Weave policy engine; live scenarios 1, 2, 3, 4, 7
4	Presenter 4	live demo B + 13 – 16	≈ 9 min	Live scenarios 5, 6, 8 and the full-run console; results, demo recap, conclusion, references, Q&A;

Slide 1 · Title (30 s)

On screen: the woven pattern on the right is our logo motif; the name Loom and its parts (Warp, Weft, Heddle, Thread, Shuttle) come from weaving.

Say

Good morning. We are Group 14: Aiman Haque, Zoya Mulani, Aariz Sheikh and Yusuf Aslam. Our major project is **Loom: a decentralised, verifying package manager for the Arch User Repository**, guided by [guide's name].

In one sentence: Loom lets you install community software from the AUR safely, without needing any cooperation from the AUR or from the people who publish packages there. It does that by containing every build, by demanding independent, publicly logged evidence before anything is installed, and by noticing when the people behind a package change.



Slide 2 · Abstract and index terms (1 min)

On screen: point at the four numbers on the right as you mention them.

Say

The abstract follows the IEEE format. The key idea is in the second sentence: community repositories run unreviewed build scripts with your full privileges, and they install whatever a maintainer publishes.

Loom is **not a malware scanner**. It combines five independent mechanisms: a least-privilege build sandbox called Heddle; Thread and Weave, which only accept an artifact when several independent rebuilders reproduce it bit-for-bit; a temporal quarantine for brand-new versions; continuity checks that notice a change of maintainer or a rewritten history; and Warp, a transparency log cosigned by independent witnesses.

On the right are four results from our own evaluation, which Presenter 4 will show in detail: nine of nine malicious cases blocked, zero of seven benign packages blocked, two hundred of two hundred split-view attacks detected, and sixty microseconds to verify an attestation signature.

Terms to be ready to explain: AUR, PKGBUILD, attestation, transparency log, witness, split view.

Slide 3 · Installing from the AUR means running a stranger's script (1 min 15 s)

On screen: trace the four steps top to bottom; steps 3 and 4 are red because that is where the damage happens.

Say

Let me show why this problem matters. On the left is what happens when you type **yay -S** some package. The helper downloads a PKGBUILD from the AUR's git server. **makepkg** then **runs that PKGBUILD, a shell script, as your own user**. Inside its build function it can read your SSH keys, your cloud credentials and your browser data, and it can open network connections. Only after that does pacman install the result as root, running the package's install hooks as root too.

Nobody reviews uploads to the AUR, nothing is signed, and when a maintainer abandons a package, **anyone can adopt it** and push a new version to every existing user. Miller et al. [6] showed that abandonment is common and that users have no plan for it. The XZ Utils backdoor in 2024 [5] is the textbook case: a patient takeover of maintainership followed by a payload hidden in the build process.

On the right, the scale, all from peer-reviewed work since 2023: 107 distinct attack vectors on open-source supply chains [1]; 24,356 malicious packages in the largest in-the-wild dataset [4]; three quarters of malicious PyPI packages reached users **through source installation** [3], which is exactly how the AUR works; and fewer than one percent of packages sandbox themselves [15]. So containment has to be enforced by the package manager. It cannot be left to package authors.

Terms to be ready to explain: orphan adoption, makepkg, pacman hooks, source installation.



Slide 4 · Problem, objectives and the five defences (1 min 15 s)

On screen: walk the five cards left to right; each has an objective at the top and the attack it counters at the bottom.

Say

Our problem statement is in the dark box: install AUR packages only when there is independent, publicly verifiable evidence that the artifact matches its source; contain every build under least privilege; and detect changes of publishing authority; all with **no cooperation** from the AUR, its maintainers or upstream. That last constraint is what makes the problem hard.

We turned that into five objectives, and each is met by one subsystem. One: contain every build, handled by **Heddle**. Two: verify, don't trust, handled by **Thread and Weave**. Three: slow down fast attacks, handled by **Quarantine**. Four: notice who changed, handled by **Continuity**. Five: make the evidence public, handled by **Warp**. A sixth objective runs through all of them: every decision must explain which rule fired and how to proceed.

The mechanisms are deliberately independent, so an attacker who beats one still faces four. The footer states what we do not claim, for example that if every rebuilder faithfully reproduces malicious source code, reproducibility alone cannot tell. We state this openly because it is exactly why we have five mechanisms and not one.

Terms to be ready to explain: threat model, least privilege, k-of-n, independent rebuilders.

Slide 5 · Literature review (1 of 2) (1 min)

On screen: don't read the table; pick the highlighted phrases.

Say

We reviewed sixteen papers, all from 2023 onwards and from venues such as IEEE S&P, ACM CCS, USENIX Security, NDSS, ASE, FSE, ICSE, MSR and DSN. Each row gives the contribution and the gap Loom fills.

The first four characterise the threat. Ladisa et al. [1] built the reference taxonomy of supply-chain attacks. Ladisa [2], Guo [3] and Zhou [4] study malicious packages; their detectors are useful but **probabilistic**, so an attacker can adapt, which is why Loom contains and verifies instead of classifying.

Przymus and Durieux [5] reconstruct the XZ Utils attack, and Miller et al. [6] study abandonment. Together they tell us the attack surface is changes of authority plus build-time payloads, which our continuity checks and sandbox target. Schorlemmer [7] and Merrill [8] cover signing: signing tells you **who** published, not whether the artifact matches its source, and it needs registry cooperation that the AUR does not offer.



Slide 6 · Literature review (2 of 2) (1 min)

Say

The second half covers the building blocks we use. Abu Ishgair et al. [9] argue for layered defences evaluated by the attacks they actually stop, which is exactly how we evaluate Loom, with an ablation study. Tamanna et al. [10] show that provenance frameworks like SLSA stall because they need the producer's buy-in; Loom produces its own provenance through independent rebuilders.

Malka, Zacchiroli and Zimmermann [11, 12] show that independent rebuilding is feasible at scale: over 700,000 Nix packages rebuilt. But nobody turns reproducibility into an install-time decision, which is what our Weave engine does.

Parakeet [13] and OPTIKS [14] show that transparency logs work in production without any blockchain; Warp applies that idea to build attestations. Finally, Alhindi and Hallett [15] found that sandboxing is rarely adopted by developers, and Brandt et al. [16] validate the Landlock-plus-seccomp combination that Heddle relies on.

Slide 7 · Research gap (1 min)

On screen: run your finger down the green Loom column, then across the bottom row.

Say

This table is the conclusion of our review. Each column is a class of existing solution and each row a property. yay and paru add convenience, not verification. A clean chroot build helps a little with isolation. Malware detectors are probabilistic. Signing needs registry cooperation. Reproducible-builds research measures reproducibility but never makes an install decision.

Only the last column has every property, and it is the only one that also **works with zero registry cooperation**. That combination, verified at install time on the client, is our contribution.

Handover line

“So that is the problem and the gap. [Presenter 2] will now show you how Loom is built: its architecture, and what happens step by step when you install a package.”



Questions the jury is likely to ask you

Answer in two or three sentences, then stop. If a question belongs to another presenter's area, say "My teammate covered that, [name] can answer" and hand it over.

Q. Why the AUR and not npm or PyPI?

The AUR is the extreme case: installing means executing the build script on your machine, nothing is signed, and orphan adoption is a built-in feature. If Loom works there it works anywhere, and the design keeps all AUR knowledge behind one interface, so npm or PyPI backends need no changes to the security engines (requirement NFR-MNT-1).

Q. How is this different from yay or paru?

yay and paru are convenience wrappers around makepkg: they fetch and build, with no sandbox, no verification and no history. Loom replaces them at the point of installation and adds all five defences.

Q. Why not just use machine-learning malware detection?

Detection is probabilistic and adversaries adapt to it [2]. Loom does not guess whether code is malicious; it confines what code can do and requires independent evidence that the artifact matches the source. The two approaches are complementary, and a detector could feed Loom's advisory input in future.

Q. Why not use signing, such as Sigstore?

Signing requires publishers or the registry to take part, and the AUR offers neither (our constraint CON-1). Signing also proves who published an artifact, not that it matches its source [7]. Loom obtains that evidence itself through independent rebuilders.

Q. What exactly is novel?

The combination, done entirely client-side without cooperation: sandboxed builds, k-of-n reproducible attestations, a witnessed log, continuity drawn from the log's own history (so first-time installers are protected too), an install-placement policy, and failing closed on hostile sandbox denials even when a build exits successfully.

Q. Are your references recent and credible?

All sixteen are from 2023 to 2026, mostly peer-reviewed at top venues: IEEE S&P, ACM CCS, USENIX Security, NDSS, ASE, ESEC/FSE, ICSE, MSR, DSN, ACSAC and HICSS. Two are recent arXiv preprints, labelled as such.

Numbers every presenter must know

Fact	Value	Where it comes from
Malicious cases blocked (AC-1)	9 / 9 (100%)	loom-eval E1, labelled incident corpus
Benign packages blocked (AC-2)	0 / 7 (0%)	loom-eval E2
Best single mechanism (AC-4)	catches 3 of 9 (sandbox)	loom-eval E4 ablation
Policy decision time	8.3 μ s (target < 500 ms)	loom-eval E5, release build
Ed25519 attestation verification	60 μ s (target < 100 ms)	loom-eval E5, release build
Split views detected (AC-6)	200 / 200	loom-eval E6
Sink hits, same malicious build	0 confined vs 1 unconfined	demo scenario 7
Code base	11 Rust crates, ~12,700 lines, 75 automated tests	cargo test --workspace
Default policy	k = 2 attestations, 2-of-3 witnesses, 72 h quarantine, continuity change = block	SRS Appendix A



Fact	Value	Where it comes from
Deployment in the demo	mock AUR :7700, Warp :7710, witnesses :7721-3, rebuilders/peers :7731-3, console :7790	demo/run.sh